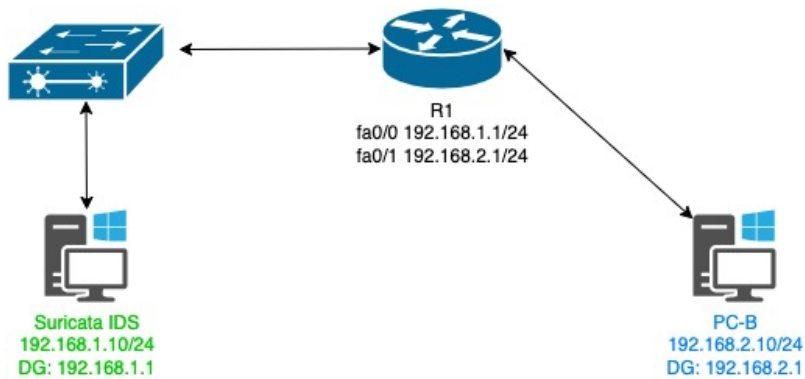


SURICATA IDS SETUP

Topology Setup



	Suricata IDS	PC-B (Blue)
Adapter 1	NAT	
Adapter 2	VMWare12	VMWare11
Adapter 2 IP	192.168.1.10	192.168.2.10
Adapter 2 Subnet	255.255.255.0	255.255.255.0
Adapter 2 Default-gateway	192.168.1.1	192.168.2.1

Build the physical topology

- Cable the physical devices/VMs as shown above
- Load R1 script (found in Moodle) into the Router and enable fa0/0 and fa0/1 interfaces
- NIC1 on Suricata IDS should already be set to NAT and have a valid IP address
- Add NIC2 to Suricata and assign the VMWare12 adapter and IP addresses shown above.
- Use the Blue VM for PC-B and assign the IP address as shown in the above table.
- Test connectivity throughout the 192.168.1.0 network using **ping** command. **Troubleshoot any connectivity issues before moving on.**
- Use terminal to find the name of NIC2, then disable NIC2 until later

. To find your interface open another terminal windows and type `ip a` and note down the interface where you set the IP address

Update Suricata VM and install prerequisite packages

Tasks:

- From the terminal update the VM and install prerequisite packages using the commands below

Commands:

```
sudo apt-get update && sudo apt-get update
sudo apt -y install libnetfilter-queue-dev libnetfilter-queue1
sudo apt -y install libnfnetlink-dev libnfnetlink0
sudo apt install curl && sudo apt install jq
```

```
# note down the name of NIC2 (probably ens33)
ip a
```

Suricata Installation

Tasks:

- Take a snapshot of the Ubuntu 24.04 VM and name the snapshot 'Base Image'
- Boot the VM and set the IP address as above
- From the terminal install additional packages adding the *Suricata* repository to our list
- Set *Suricata* to start on bootup

Commands:

```
sudo apt-get install software-properties-common
sudo add-apt-repository ppa:oisf/suricata-stable
sudo apt-get install suricata -y

sudo systemctl enable suricata.service
```

Check status/start/stop

Tasks:

- Check the status of Suricata to ensure it successfully installed. Not it might show as *failed* to run at this stage which is expected since we have not configured the IDS, however, this shows us that the IDS was successfully installed.

- Stop the IDS running so we can progress to configure it

Commands:

```
sudo systemctl status suricata.service
sudo systemctl stop suricata
```

Configure Suricata

Tasks:

- *Suricata* configuration files and rules are stored in */etc/suricata* and */etc/suricata/rules* respectively. Start by listing these files to ensure they are present
- Next configure *Suricata* by editing *suricata.yaml*
- Set the appropriate variables listed below to set internal and external Ips and interface. Note you can use *Ctrl-w* to search for text within the file.
- The *communit-id* is used for event correlation. It can be useful when using tools like **zeek** or importing logs in json format. When enabled, Suricata will create records so they can be matched to events in zeek. The seed needs to be unique.
- Under *Default-rule-path* you will find the file where Suricata rules are stored (we will download these later). Specify a new file called *local.rules* that we will use for custom rules that we will create. We will need to create this file later in directory */var/lib/suricata/rules*
- Save the **yaml** file and exit (press *CTRL+X*, *Y*, then *enter*)

Commands:

Note: the *yaml* file listed in this directory is the *suricata* configuration file

```
sudo ls -la /etc/suricata
```

Note: below rules directories are set of prepackaged rules for *suricata*. This may not be created at this stage so do not worry if it says no such file or directory exists at the moment

```
sudo ls -la /etc/suricata/rules
sudo ls -la /var/lib/suricata/rules
```

edit the *yaml* file and set the below variables. Use *CTRL-W* to search within file

```
sudo nano -c /etc/suricata/suricata.yaml
```

amend *HOME_NET* and check *EXTERNAL_NET* is *!HOME_NET*

Vars:

```
HOME_NET "[x.x.x.x/24]"
EXTERNAL_NET "[!HOME_NET]"
```

```
# amend to "[192.168.1.0/24]"
# check this is correct
```

```

# amend to the interface you recorded earlier
af-packet:
  Interface: xxx                                # probably ens33

pcap:
  Interface: xxx                                # probably ens33

# amend community from false to true
Community-id: false                            # amend to true

# search for the default-rule-path and specify a local.rules file

Default-rule-path: /var/lib/suricata/rules
Rule-files:
- suricata.rules
- local.rules                                # add local.rules –we will create this later

# Save and exit yaml file

```

Update Suricata configuration

Tasks:

- Start by downloading the current maintained rules list for Suricata. These are fetched from <http://emergingthreats.net>
- List further sources for downloading additional rules and specify to use these as required. Some of these will require a subscription. In the example command below it would pull down additional rules related to windows malware from a different repository from the default Suricata location
- Update Suricata to use the new rules

Commands:

```
# ignore any ERROR about protocols that are not enabled – we are not using them
```

```
sudo suricata-update
```

```

# notice the suricata.rules exists but local.rules does not yet. We will create this later
sudo ls -la /var/lib/suricata/rules

```

list all possible rules packs we can use

```
sudo suricata-update list-sources
```

enable a few rules pack for use

```
sudo suricata-update enable-source malsilo/win-malware
```

```
sudo suricata-update enable-source tgreen/hunting
```

update suricata with new rules and allow suricata to run a self-test (**suricata -T**)

```
sudo suricata-update
```

View suricata maintained rules that have just been updated

Tasks:

- Start by viewing the *suricata.rules* file timestamp to ensure it has updated.
- Switch to the root user to gain necessary permission to view rules files
- Verify you are using the root user account
- View all the rules *suricata* can now use to detect threats

Commands:

check file timestamp is recent

```
sudo ls -la /var/lib/suricata/rules/
```

switch to root user

```
sudo su
```

```
whoami
```

move into rules directory

```
cd /var/lib/suricata/rules/
```

```
ls
```

```
cat suricata.rules
```

```
exit
```

Test Suricata configuration

Tasks:

- Test the newly configured suricata functions correctly.
- Note: Suricata can be run as a background Daemon using *-D*
- We can also run Suricata by specifying configuration file and rules files as shown below. When doing this replace the variables e.g. replace *eth0* with your interface
- (Optional) Default way to run Suricata with the normal rule set

Commands:

Here -T tells suricata we want to test the configuration and -v runs verbose.

Suricata will output log files (*fast.log* & *eve.json*) in */var/log/suricata*

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
```

ignore warning that *local.rules* is not present - we haven't created the file yet

if threads are created on step two this means it is working so CTRL-C to stop

```
sudo suricata -c /etc/suricata/suricata.yaml -s signatures.rules -i ens33
```

#suricata should now be fully functional (shown as green)

```
sudo systemctl start suricata.service
```

```
sudo systemctl status suricata.service
```

Test Suricata maintained rules

Tasks:

- We are now ready to test suricata when running live.
- We can use curl to request a page. This test facility is designed to return the request as *root* user simulating an attack as though someone has got root access

Commands:

should return a root username

```
curl http://testmyids.org/uid/index.html
```

this should an entry in the log file for the test we just did. If it is blank try below

```
sudo cat /var/log/suricata/fast.log
```

check data is present at this stage as it is too difficult to find out event in the data

```
sudo cat /var/log/suricata/eve.json
```

Create custom rule

Tasks:

- We now know that the maintained Suricata rule set we downloaded earlier works. We can proceed to create some custom rules that would be unique to our installation i.e. an organisations specific network setup they are trying to protect
- First, stop Suricata running and create a simple rule to detect ICMP echo requests

```
alert icmp any any -> $HOME_NET any (msg:"ICMP External Ping"; sid:1; rev:1;)
```

This rule will log any pings coming from an external network to our Home network using any port

SID is a signature id we can specify 1 as it is our first custom rule

REV is revision which again we set to 1

- Test the configuration as we did before and restart Suricata
- Start Suricata running

Commands:

```
sudo systemctl stop suricata.service  
sudo nano /var/lib/suricata/rules/local.rules
```

```
alert icmp any any -> $HOME_NET any (msg:"ICMP External Ping"; sid:1; rev:1;)
```

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v  
sudo systemctl start suricata.service
```

Test new custom rule

Tasks:

- Test the new custom rule by **pinging Suricata from PC-B on the external network and from the router**
- Check both the *fast.log* and *eve.json* log files for this activity

Commands:

```
sudo cat /var/log/suricata/fast.log
```

```
sudo tail -f /var/log/suricata/eve.json | jq 'select(.event_type=="alert")'
```

- Take a snapshot of the Ubuntu 24.04 VM and name the snapshot 'Suricata Install' you can return to this in the future if needed.

OPTIONAL (If time permits)

Tasks:

- Research how to write a Suricata rule to detect a suspected ssh connectivity.
- Setup the necessary configuration on the physical equipment
- Simulate malicious ssh activity, and test the newly created rule.